

Ejercicios Básicos de Laravel

Bloque 1: Comandos Artisan

Ejercicio 1.1: Primeros pasos con Artisan

Objetivo: Familiarizarse con los comandos básicos de Artisan.

Tareas:

1. Lista todos los comandos disponibles en tu instalación de Laravel.
2. Consulta la ayuda del comando `make:controller`.
3. Genera una clave de aplicación para tu proyecto.
4. Lista todas las rutas definidas en tu aplicación (aunque aún no hayas creado ninguna).

Entregable: Capturas de pantalla de la ejecución de cada comando.

Ejercicio 1.2: Creación de componentes

Objetivo: Crear la estructura básica de un módulo con Artisan.

Contexto: Vas a crear un módulo para gestionar productos en una tienda online.

Tareas:

1. Crea un controlador llamado `ProductController`.
2. Crea un modelo llamado `Product`.
3. Crea una migración llamada `create_products_table`.
4. Crea un seeder llamado `ProductSeeder`.

Entregable:

- Comandos ejecutados.
 - Captura del árbol de directorios mostrando los archivos creados.
-

Bloque 2: Enrutamiento

Ejercicio 2.1: Rutas básicas

Objetivo: Practicar la definición de rutas simples.

Tareas:

1. En el archivo `routes/web.php`, crea una ruta GET para `/inicio` que devuelva el texto "Bienvenido a mi aplicación Laravel".
2. Crea una ruta GET para `/sobre-nosotros` que devuelva un texto descriptivo de tu aplicación.
3. Crea una ruta GET para `/contacto` que devuelva "Página de contacto".
4. Prueba todas las rutas en el navegador.

Entregable: Código del archivo `routes/web.php` y capturas de las rutas funcionando.

Ejercicio 2.2: Rutas con parámetros

Objetivo: Trabajar con parámetros en las rutas.

Tareas:

1. Crea una ruta `/usuario/{nombre}` que muestre "Hola, [nombre]".
2. Crea una ruta `/producto/{id}` que muestre "Detalles del producto ID: [id]".
3. Crea una ruta `/blog/{categoria}/{id}` que muestre "Mostrando artículo [id] de la categoría [categoria]".
4. Crea una ruta `/saludo/{nombre?}` donde el nombre sea opcional. Si no se proporciona, debe mostrar "Hola, invitado".

Entregable: Código de las rutas y capturas probando diferentes valores de parámetros.

Ejercicio 2.3: Rutas con nombre y verbos HTTP

Objetivo: Dominar las rutas nombradas y los diferentes verbos HTTP.

Tareas:

1. Crea las siguientes rutas con sus respectivos nombres:
 - GET `/productos` → nombre: `products.index`
 - GET `/productos/crear` → nombre: `products.create`
 - POST `/productos` → nombre: `products.store`
 - GET `/productos/{id}` → nombre: `products.show`
2. Crea una ruta GET `/prueba-rutas` que devuelva enlaces HTML usando la función `route()` para cada una de las rutas anteriores.
3. Ejecuta `php artisan route:list` y verifica que todas tus rutas aparecen con sus nombres correctos.

Entregable: Código de las rutas y captura del resultado de `route:list`.

Ejercicio 2.4: Orden de rutas (Detectar el error)

Objetivo: Entender la importancia del orden de las rutas.

Tareas:

1. Define estas rutas EN ESTE ORDEN:

```
Route::get('/producto/{nombre}', function($nombre) {  
    return "Producto: $nombre";  
});  
Route::get('/producto/nuevo', function() {  
    return "Formulario de nuevo producto";  
});
```

2. Accede a `/producto/nuevo` y anota qué sucede.
3. Explica por qué ocurre este comportamiento.
4. Corrige el problema reordenando las rutas.
5. Verifica que ahora funciona correctamente.

Entregable:

- Explicación del problema.
- Código corregido.

- Capturas antes y después de la corrección.
-

Bloque 3: Controladores

Ejercicio 3.1: Controlador básico

Objetivo: Crear y usar un controlador básico.

Tareas:

1. Crea un controlador llamado `PageController` usando Artisan.
2. Añade los siguientes métodos al controlador:
 - `home()` : debe devolver "Página de inicio"
 - `about()` : debe devolver "Acerca de nosotros"
 - `contact()` : debe devolver "Página de contacto"
3. Modifica el archivo de rutas para que las URLs `/` , `/acerca` y `/contacto` apunten a estos métodos.
4. Asigna nombres a estas rutas: `home` , `about` y `contact` .

Entregable:

- Código del controlador.
 - Código de las rutas.
 - Capturas de las páginas funcionando.
-

Ejercicio 3.2: Controlador con parámetros

Objetivo: Manejar parámetros en los controladores.

Tareas:

1. Crea un controlador `UserController` .
2. Añade un método `show($id)` que devuelva "Mostrando usuario con ID: [id]".
3. Añade un método `profile($username)` que devuelva "Perfil de usuario: [username]".
4. Crea las rutas correspondientes:
 - `/usuario/{id}` → `UserController@show`
 - `/perfil/{username}` → `UserController@profile`

Entregable: Código del controlador y las rutas con capturas de prueba.

Ejercicio 3.3: Controlador RESTful

Objetivo: Crear un controlador de recursos completo.

Tareas:

1. Crea un controlador resource llamado `BookController` usando el flag `--resource` .
2. Examina los métodos que se han creado automáticamente.
3. Implementa cada método con una respuesta simple:
 - `index()` : "Listado de todos los libros"
 - `create()` : "Formulario para crear libro"
 - `store()` : "Guardando nuevo libro"
 - `show($id)` : "Mostrando libro con ID: [id]"

- `edit($id)` : "Formulario para editar libro [id]"
- `update($id)` : "Actualizando libro [id]"
- `destroy($id)` : "Eliminando libro [id]"

4. Define la ruta resource correspondiente en una sola línea.

5. Ejecuta `php artisan route:list` y verifica todas las rutas creadas.

Entregable:

- Código del controlador.
- Línea de la ruta resource.
- Captura de `route:list` mostrando las 7 rutas.
- Capturas probando al menos 4 rutas diferentes.

Ejercicio 3.4: Controlador API

Objetivo: Crear un controlador para una API REST.

Tareas:

1. Crea un controlador API llamado `ArticleController` con el flag `--api`.
2. Observa que no tiene los métodos `create()` ni `edit()`. Explica por qué.
3. Implementa cada método devolviendo un array (que Laravel convertirá a JSON):
 - `index()` : ['message' => 'Listado de artículos']
 - `store()` : ['message' => 'Artículo creado']
 - `show($id)` : ['message' => 'Artículo ID: ' . \$id]
 - `update($id)` : ['message' => 'Artículo actualizado']
 - `destroy($id)` : ['message' => 'Artículo eliminado']
4. Define la ruta `apiResource` en `routes/api.php`.
5. Prueba las rutas (recuerda que las rutas de `api.php` llevan el prefijo `/api/`).

Entregable:

- Código del controlador.
- Explicación sobre la ausencia de `create()` y `edit()`.
- Capturas mostrando las respuestas JSON.

Bloque 4: Vistas con Blade

Ejercicio 4.1: Primera vista

Objetivo: Crear y cargar una vista desde un controlador.

Tareas:

1. Crea una vista en `/resources/views/welcome.blade.php` con un HTML básico que diga "Bienvenido a Laravel".
2. Crea un controlador `WelcomeController` con un método `index()`.
3. El método debe cargar y devolver la vista `welcome`.
4. Crea la ruta correspondiente.

Entregable:

- Código de la vista.

- Código del controlador.
 - Captura de la vista renderizada en el navegador.
-

Ejercicio 4.2: Pasar datos a vistas

Objetivo: Enviar datos desde el controlador a la vista.

Tareas:

1. Crea un controlador `StudentController` con un método `show($name)` .
2. El método debe pasar el nombre del estudiante a una vista.
3. Crea la vista `/resources/views/student/profile.blade.php` .
4. La vista debe mostrar: "Perfil del estudiante: [nombre]" usando la sintaxis de Blade `{{ $nombre }}` .
5. Crea la ruta `/estudiante/{nombre}` .

Extra: Pasa además la edad y la ciudad del estudiante a la vista y muéstralos.

Entregable:

- Código del controlador.
 - Código de la vista.
 - Capturas con diferentes nombres de estudiantes.
-

Ejercicio 4.3: Estructuras de control en Blade

Objetivo: Usar condicionales y bucles en las vistas.

Tareas:

1. Crea un controlador `ProductController` con un método `list()` .
2. El método debe pasar un array de productos a la vista:

```
$products = [  
    ['name' => 'Laptop', 'price' => 999, 'stock' => 5],  
    ['name' => 'Mouse', 'price' => 25, 'stock' => 0],  
    ['name' => 'Teclado', 'price' => 45, 'stock' => 10],  
];
```

3. Crea la vista `/resources/views/products/list.blade.php` .
4. En la vista, usa `@foreach` para mostrar todos los productos en una tabla HTML.
5. Usa `@if` para mostrar "SIN STOCK" en rojo cuando el stock sea 0.
6. Usa `@if` para mostrar "BAJO STOCK" en naranja cuando el stock sea menor que 5.

Entregable:

- Código del controlador.
 - Código de la vista con las estructuras de control.
 - Captura de la tabla renderizada.
-

Bloque 5: Proyecto Integrador

Ejercicio 5.1: Sistema de gestión de tareas (TODO List)

Objetivo: Crear un CRUD completo integrando todos los conceptos aprendidos.

Descripción: Vas a crear un sistema para gestionar tareas pendientes.

Requisitos:

1. **Estructura de datos:** Cada tarea debe tener:

- ID (automático)
- Título
- Descripción
- Estado (pendiente/completada)
- Fecha de creación

2. **Controlador:**

- Crea un `TaskController` de tipo resource.
- Implementa todos los métodos necesarios.

3. **Rutas:**

- Define la ruta resource para las tareas.
- Todas las rutas deben tener nombres apropiados.

4. **Vistas** (crea estas vistas en `/resources/views/tasks/`):

- `index.blade.php` : Lista todas las tareas en una tabla.
- `create.blade.php` : Formulario para crear una nueva tarea.
- `show.blade.php` : Muestra los detalles de una tarea.
- `edit.blade.php` : Formulario para editar una tarea existente.

5. **Funcionalidades:**

- Por ahora, usa un array en memoria para simular la base de datos.
- Lista de tareas con botones de ver, editar y eliminar.
- Crear nueva tarea.
- Ver detalles de una tarea.
- Editar una tarea existente.
- Eliminar una tarea (con confirmación).
- Las tareas completadas deben mostrarse tachadas o en un color diferente.

Entregable:

- Código completo del controlador.
- Código de todas las rutas.
- Código de todas las vistas.
- Documento explicando el flujo de la aplicación.
- Video o capturas mostrando todas las funcionalidades.

Bonus:

- Añade un contador que muestre cuántas tareas están pendientes y cuántas completadas.
- Implementa un buscador de tareas por título.
- Añade validación de formularios (el título no puede estar vacío).

Ejercicio 5.2: Blog personal

Objetivo: Crear un blog básico con artículos y categorías.

Descripción: Sistema de blog con múltiples recursos relacionados.

Requisitos:

1. Estructura:

- **Artículos:** título, contenido, autor, fecha, categoría.
- **Categorías:** nombre, descripción.

2. Controladores:

- `ArticleController` (resource).
- `CategoryController` (resource).
- `HomeController` (para la página principal).

3. Rutas:

- Rutas resource para artículos y categorías.
- Ruta `/` para la página de inicio.
- Ruta `/categoria/{nombre}` para filtrar artículos por categoría.

4. Vistas:

- Página de inicio mostrando los últimos 5 artículos.
- Lista completa de artículos.
- Vista de detalle de un artículo.
- Lista de categorías.
- Artículos filtrados por categoría.

5. Funcionalidades:

- CRUD completo de artículos.
- CRUD completo de categorías.
- Filtrar artículos por categoría.
- Cada artículo debe mostrar su categoría.
- Navegación clara entre todas las secciones.

Entregable:

- Código completo de controladores.
- Archivo de rutas completo.
- Todas las vistas creadas.
- Diagrama del flujo de navegación.
- Memoria del proyecto explicando las decisiones de diseño.

Bonus:

- Implementa un sistema de búsqueda de artículos.
- Añade paginación (simulada) mostrando 5 artículos por página.
- Crea una barra lateral con las categorías más usadas.

Bloque 6: Depuración y Buenas Prácticas

Ejercicio 6.1: Debugging con `route:list`

Objetivo: Practicar la depuración de problemas de enrutamiento.

Tareas:

1. Crea 10 rutas diferentes con nombres variados.
2. Ejecuta `php artisan route:list`.
3. Identifica cuáles son rutas GET, POST, PUT, etc.
4. Identifica cuáles tienen parámetros obligatorios u opcionales.
5. Crea un documento que explique cada ruta de tu aplicación.

Entregable: Captura de `route:list` y documento explicativo.

Ejercicio 6.2: Refactorización

Objetivo: Mejorar código siguiendo las convenciones de Laravel.

Código inicial (en `routes/web.php`):

```
Route::get('/user-list', function() {  
    return "List of users";  
});  
  
Route::get('/user-details/{id}', function($id) {  
    return "User ID: $id";  
});  
  
Route::get('/create-user', function() {  
    return "Create user form";  
});
```

Tareas:

1. Crea un controlador apropiado para estas rutas.
2. Mueve la lógica al controlador.
3. Renombra las URLs siguiendo las convenciones REST.
4. Añade nombres a las rutas.
5. Documenta los cambios realizados.

Entregable:

- Código refactorizado.
 - Documento explicando cada mejora.
-